Best Seat

AlgoExpert · Medium · Arrays

The problem

- A row of seats (1 taken, 0 free; both ends taken).
- Return the index of the free seat with the most space on both sides (centre of the widest gap, leftmost on ties), or -1.

Intuition

- The best seat is the middle of the longest run of empty seats between occupied ones.

Brute force vs optimal

- Brute force: for each empty seat, scan both ways for neighbours → $O(n^2)$
- Optimal: find runs of zeros in one pass → $O(n)$
- Middle of the longest run is the answer

Approach

- 1. Scan for runs of zeros between ones
- 2. Track the longest run
- 3. Return its middle index, or minus one if none

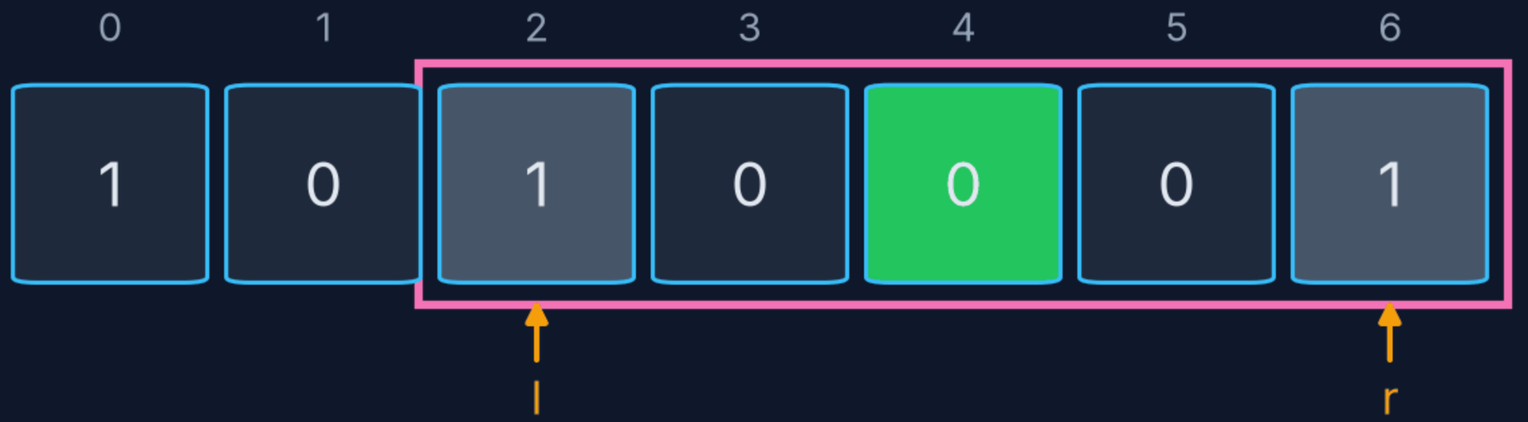
Worked example




```
gap = 1
best seat = 1
```

Run of 1 empty seat(s) between 0 and 2 → best 1

Worked example




```
gap = 3
best seat = 4
```

Run of 3 empty seat(s) between 2 and 6 → best 4

Java solution

```java
public class BestSeat {
    public static int bestSeat(int[] seats) {
        int best = -1, bestGap = 0, l = 0;
        while (l < seats.length) {
            int r = l + 1;
            while (r < seats.length && seats[r] == 0) r++;
            int gap = r - l - 1;
            if (gap > bestGap) { bestGap = gap; best = (l + r) / 2; }
            l = r;
        }
        return best;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- No empty seat → −1
- Ties: keep the leftmost (strictly greater)
- Even-length runs: (l + r) / 2 picks the left middle

Interview tips & follow-ups

- Clarify ends: seats at both ends are occupied
- Follow-up: seat many people one by one (heap of gaps)

Recap

- The best seat is the middle of the longest run of empty seats between occupied ones.
- Complexity: O(n)